

Automated Compilation Pipelines - XLA

```
import tensorflow as tf

# Simple model component
class Block(tf.Module):
    def __init__(self, c): super().__init__(); self.conv =
tf.keras.layers.Conv2D(c, 3, padding="same")
    def __call__(self, x, training=False):
        x = self.conv(x)
        x = tf.keras.layers.BatchNormalization()(x,
training=training)
        return tf.nn.relu(x)

# Enable XLA for this function (TF >= 2.11: jit_compile=True)
@tf.function(jit_compile=True)
def step(x):
    b = Block(32)
    return b(x, training=False)

# Warm up + run
x = tf.random.normal([1, 224, 224, 3])
y = step(x) # XLA compiles on first call; later calls use
optimized code
```

What XLA (Accelerated Linear Algebra) does

- Fuses ops into single kernels
- Picks efficient layouts (e.g., NHWC ↔ NCHW)
- Reuses buffers to cut memory

When to use

- TensorFlow models with stable shapes
- Training or inference on CPU/GPU/TPU

When to be careful

- Highly dynamic Python control flow
- Ragged tensors or frequent shape changes

How to enable

- `@tf.function(jit_compile=True)` in TF 2.11+ (or `experimental_compile=True` in older TF)
- Or set `TF_XLA_FLAGS / tf.config.optimizer.set_jit(True)`

Hardware-Specific Optimizations – GPU (cont)

GPU Optimizations

Example: For a convolutional layer, prefer **channels_last (NHWC)** over **channels_first (NCHW)** on NVIDIA GPUs.

In a Tensor with Batch size (N) = 1 (single image), Height (H) = 2, Width (W) = 2, Channels (C) = 3 (RGB)

Then the data for one image (2x2 RGB) is:

Pixel (0,0): [R00, G00, B00]

Pixel (0,1): [R01, G01, B01]

Pixel (1,0): [R10, G10, B10]

Pixel (1,1): [R11, G11, B11]

For a 2D convolution:

- In NHWC, threads processing a 3x3 patch load contiguous pixels (good locality).
- In NCHW, threads fetching the same patch scatter across memory (poor locality) – because channels are stored separately ([all R][all G][all B]).

Tensor Processing Units (TPUs)

Google's Specialized AI Architecture

Systolic Array Design:

TPUs implement large 2D arrays of processing elements where data flows rhythmically through the grid:

```
// Conceptual systolic array operation
for (int cycle = 0; cycle < matrix_size; cycle++) {
// Each PE performs: accumulator += input_from_top * weight_from_left
// Data flows: inputs downward, weights rightward, outputs collected
PE[i][j].accumulate(input_from_above[i], weight_from_left[j]);

// Shift data for next cycle
shift_inputs_down();
shift_weights_right();
}
```

TPU Characteristics:

- **Large Matrix Units:** 128×128 or 256×256 systolic arrays
- **Mixed Precision:** BF16 computation with FP32 accumulation
- **Software Stack:** XLA compiler for automatic optimization
- **High Throughput:** Optimized for training large models

Design Trade-offs:

- Excellent for large matrix operations
- Less flexible than GPUs for irregular computations
- Requires compiler optimization for efficiency



Google's Tensor Processing Unit (TPU)
Image Source: unwire.pro

Systolic Array Deep Dive

Understanding Specialized Matrix Computation

Systolic Array Principles:

Data flows through a 2D grid of processing elements in a synchronized, rhythmic pattern

Advantages:

- **High Utilization:** All PEs active simultaneously during steady state
- **Predictable Data Flow:** No complex routing or scheduling
- **Scalability:** Linear scaling with array size
- **Energy Efficiency:** Minimal control overhead

Limitations:

- **Matrix Size Constraints:** Optimal for specific dimension ranges
- **Setup Overhead:** Filling pipeline takes time
- **Limited Flexibility:** Primarily optimized for dense matrix operations

Applications:

- Convolutional layers (after im2col transformation)
- Fully connected layers
- Attention mechanisms in transformers

Reminder from Week 3: Systolic Arrays use Tiling

```
// PE (Processing Element) operation each cycle
class ProcessingElement {
    int32_t accumulator;

    void cycle(int8_t input_from_north, int8_t weight_from_west) {
        // Multiply-accumulate operation
        accumulator += input_from_north * weight_from_west;

        // Pass data to neighbors
        send_to_south(input_from_north);
        send_to_east(weight_from_west);
    }
};
```